GOVERNOR-OT

Full Architecture & Working Flow

Layer structure, pipeline flow, the AST safety check, execution dispatch, and the real backend data flow

1. Full Layer Architecture

The system organized into its five real layers: presentation and monitoring, multi-agent orchestration, execution, the testbed and target scenarios, and the underlying platform and operations infrastructure. Each box inside a layer is a real, separate component, not a label.

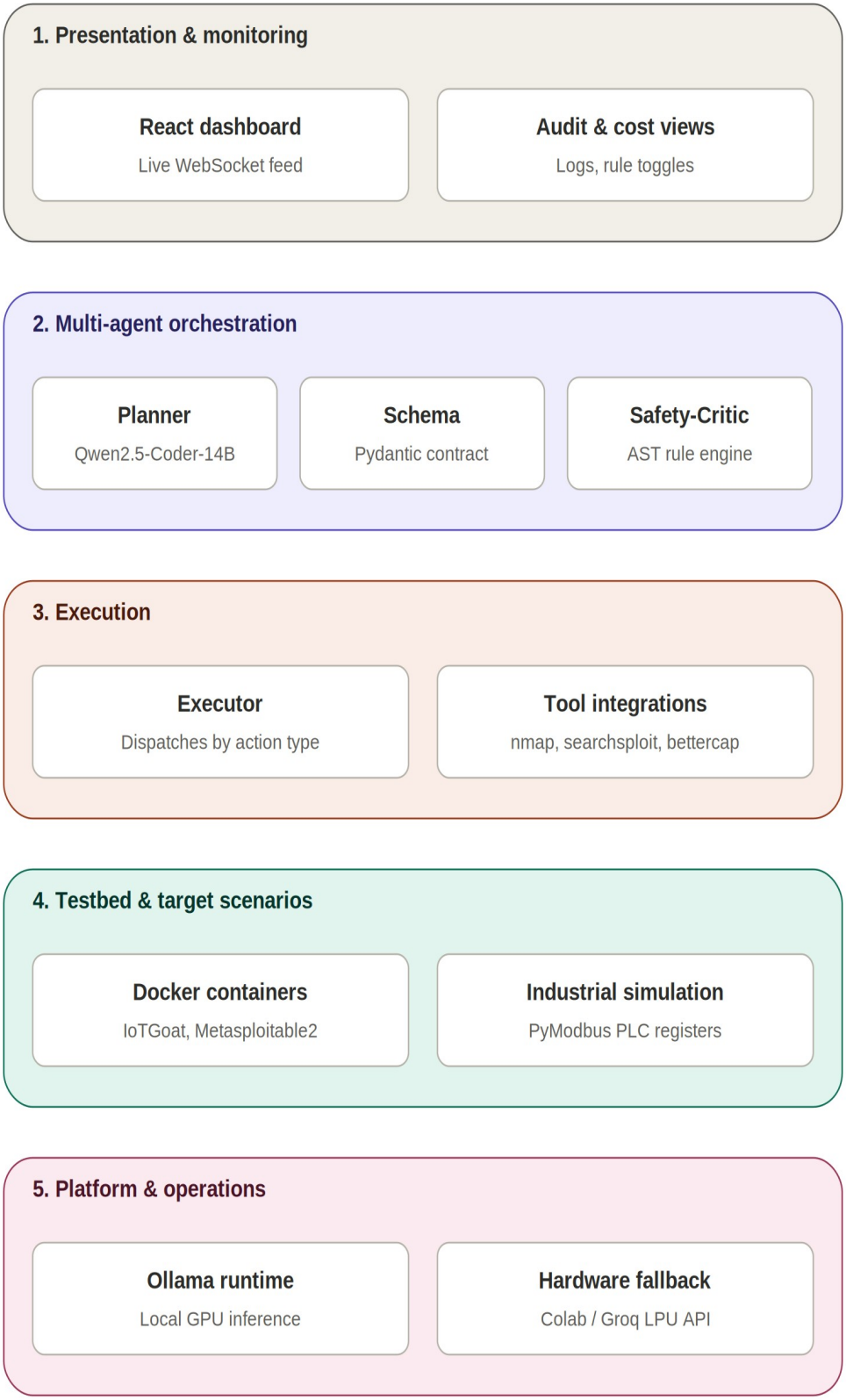


Figure 1 — the five layers and their real components

2. The Working Pipeline

How one command actually moves through the system, top to bottom, with exactly one loop: a blocked command returns to the Planner with the reason attached, rather than being discarded.

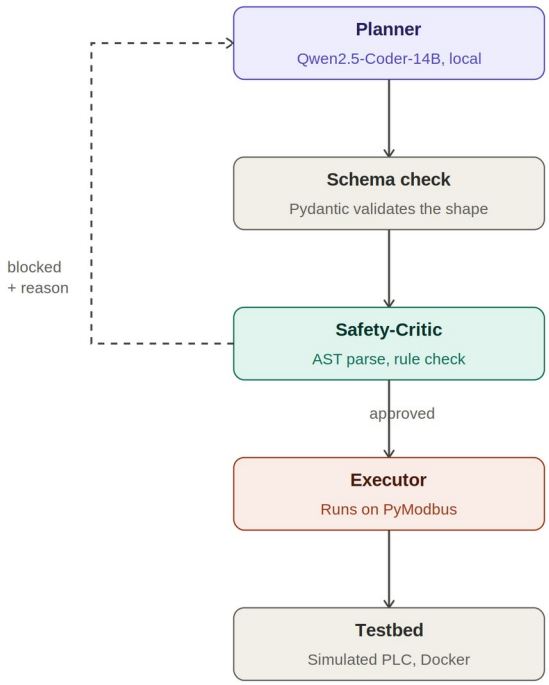


Figure 2 — Planner to Testbed, with the correction loop

3. Inside the Safety-Critic — the AST Step

What happens specifically inside the Safety-Critic box from Figure 2: the schema-validated command's expression field is parsed into a syntax tree, that tree is walked to read the function being called and its arguments, and rule functions check that against the fixed rule set, branching only at the very last step into approved or blocked.

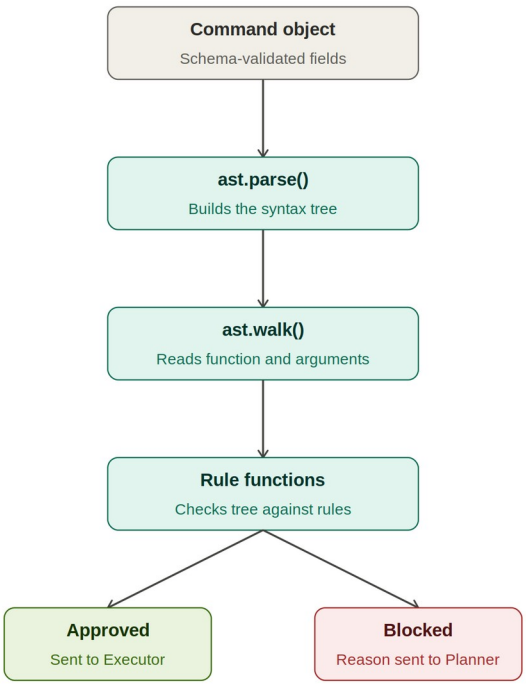


Figure 3 — ast.parse(), ast.walk(), and the rule check

4. Execution Dispatch — Where Commands Actually Go

Once a command is approved, the Executor reads its action field and branches: a Modbus read or write calls PyModbus directly against the simulated PLC, while a reconnaissance or exploit-style action shells out to the matching offensive tool against the Docker testbed.

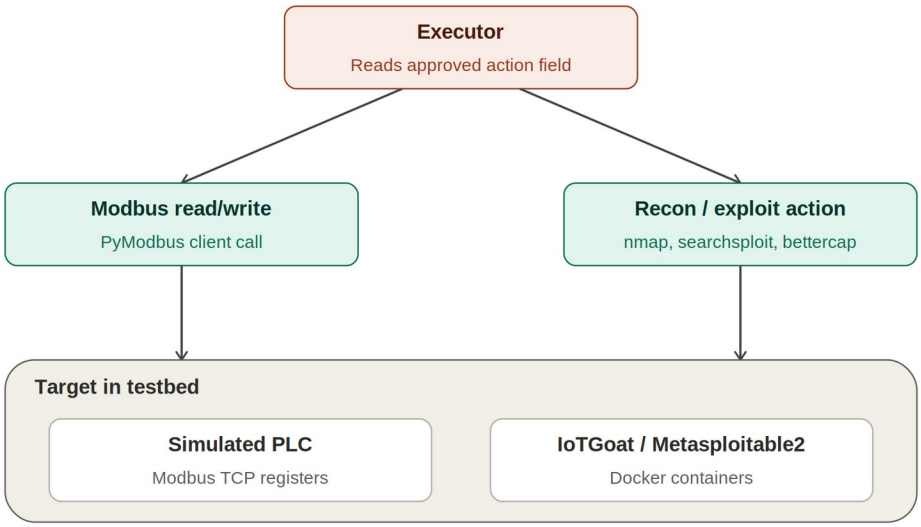


Figure 4 — the Executor's two real dispatch paths

5. The Real Backend Data Flow

This is the part no box-and-arrow diagram shows on its own: what data actually exists, in what shape, at every boundary above.

Planner to Schema: Ollama returns raw JSON text constrained to the Pydantic shape. The code calls something like Command.model_validate_json() on it — if any field is wrong, this throws immediately, before anything else runs.

Schema to Safety-Critic: a validated Command object, plus its code-like expression field, is passed in. The Critic calls ast.parse() on that expression string specifically, producing a tree it then walks with ast.walk().

Safety-Critic to Executor: the same Command object, unchanged, plus a boolean verdict. The Executor branches on command.action — a plain if or match statement — calling either a PyModbus method (client.write_register(address, value)) or starting a subprocess running the matching CLI tool, depending on the action type.

Executor to Logbook: the original command, the verdict, and whatever came back from the real call (a PyModbus response object, or a tool's stdout) get serialized together into one row. Because the schema was clean from the start, nothing has to be reshaped to store it.

Everything to Dashboard: the same logbook row, the instant it is written, is pushed out over the WebSocket as JSON. The dashboard never queries anything — it only renders whatever arrives.

— End of document —